

| 859. Kruskal算法求最小生成树

| [859 - Kruskal算法求最小生成树](#)

| 题目描述

| (请在此处粘贴题目描述)

给定一个 n 个点 m 条边的无向图，图中可能存在重边和自环，边权可能为负数。

求最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

给定一张边带权的无向图 $G=(V,E)$ ，其中 V 表示图中点的集合， E 表示图中边的集合， $n=|V|$ ， $m=|E|$ 。

由 V 中的全部 n 个顶点和 E 中 $n-1$ 条边构成的无向连通子图被称为 G 的一棵生成树，其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。

| 输入格式

第一行包含两个整数 n 和 m 。

接下来 m 行，每行包含三个整数 u,v,w ，表示点 u 和点 v 之间存在一条权值为 w 的边。

| 输出格式

共一行，若存在最小生成树，则输出一个整数，表示最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

| 数据范围

$$1 \leq n \leq 10^5,$$

$$1 \leq m \leq 2 * 10^5,$$

图中涉及边的边权的绝对值均不超过 1000。

输入样例：

```
4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4
```

输出样例：

```
6
```

💡 解题思路

Kruskal 算法（最小生成树）

- **适用场景**：适用于**稀疏图**，无需邻接矩阵或邻接表，直接用结构体存储边即可。

核心步骤

- ① 将所有边按权重从小到大排序 // 时间复杂度 $O(m \log m)$, m 为边数
- ② 枚举每条边 (a, b) ，权重为 c ：
 if a 和 b 不连通（并查集判断）
 将这条边加入最小生成树集合

关键说明

- **存储方式**：直接定义结构体存储边的两个端点和权值，例如：

```
struct Edge {
    int a, b, w;
    bool operator<(const Edge &other) const {
        return w < other.w;
    }
}
```

```
};  
Edge edges[M];
```

- **并查集作用**：用于快速判断两个点是否连通，以及合并两个连通块，是 Kruskal 算法的核心辅助结构。
- **时间复杂度**：排序占主导，为 $O(m \log m)$ ，后续并查集操作近似 $O(1)$ ，整体效率高于朴素 Prim 算法。

代码实现

my answer

```
#include <iostream>  
#include <algorithm>  
#include <cstring>  
  
using namespace std;  
const int N = 100010, M = 200010;  
int n, m;  
int p[N]; // 并查集数组  
  
struct Edge {  
    int a, b, w;  
    bool operator<(const Edge &e) const {  
        return w < e.w; // 自定义排序规则按照边权从小到大排序  
    }  
}edges[M];  
  
bool cmp(const Edge &a, const Edge &b) {  
    return a.w < b.w;  
}  
  
// 并查集压缩路径  
int find(int x)  
{  
    if (p[x] != x) p[x] = find(p[x]);  
    return p[x];  
}
```

```

void kruskal()
{
    int res = 0, cnt = 0;
    for (auto &edge : edges) {
        int a = edge.a, b = edge.b, w = edge.w;

        a = find(a), b = find(b); // 找到祖宗节点
        if (a != b) {
            // 不在一个集合
            p[a] = b; // 合并
            cnt += 1;
            res += w;
        }
    }

    if (cnt < n - 1) puts("impossible");
    else printf("%d\n", res);
}

int main()
{
    scanf("%d%d", &n, &m);

    // 并查集数组初始化
    for (int i = 1; i <= n; i++) p[i] = i;

    for (int i = 0; i < m; i++) {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        edges[i] = {a, b, w};
    }

    // 将边按照权值从小到大排序
    // sort(edges, edges + m); // 注意是边的个数
    sort(edges, edges + m, cmp);

    kruskal();

    return 0;
}

```

- **时间复杂度**: $O(m \log m)$
- **空间复杂度**: $O(n + m)$

to optimize

- **时间复杂度**: $O(\dots)$
- **空间复杂度**: $O(\dots)$

🌟 tips

others' answer

- **时间复杂度**: $O(\dots)$
- **空间复杂度**: $O(\dots)$

🌟 tips

复盘与扩展

关键点:

易错点:

相关题目

🕒 本次耗时: ____ 分钟